# SIMATS ENGINEERING

## CO2_AT3 Error Analysis Task

**COURSE NAME: Software Testing** **COURSE CODE: CSA3712**

**COURSE OUTCOME COVERED**

**CO2:** *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

| Sl. No | Questions | Bloom's Level |
|---|---|---|
| 1 | A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness. | 3 |
| 2 | A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability. | 3 |
| 3 | A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection. | 5 |

| | | |
|---|---|---|
| 4 | A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection. | 5 |
| 5 | A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation. | 3 |
| 6 | A developer claims full Condition Coverage for a complex `if(A && B | |
| 7 | A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects. | 3 |
| 8 | During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity. | 5 |
| 9 | An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance. | 5 |

| | | |
|---|---|---|
| 10 | A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy. | 5 |

***Code of Conduct:***

*I **Hemanth Kumar B(192421263)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student***
**Hemanth Kumar B**

## RUBRICS FOR CALCULATION:

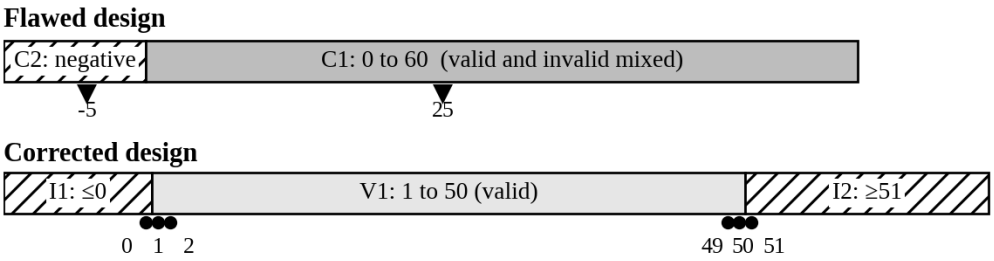
| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Error Identification | 20% | Accurately identifies all errors with clear understanding of issues | Identifies most errors with minor omissions | Identifies limited errors; some are missed | Fails to identify key errors |
| Root Cause Analysis | 30% | Clearly explains underlying causes with strong technical reasoning | Explains causes with moderate clarity | Partial explanation; weak reasoning | Unable to identify causes of errors |
| Correction & Justification | 25% | Provides correct solutions with strong justification and logic | Provides correct corrections with some justification | Corrections are partially correct or weakly justified | Incorrect or no corrections provided |
| Application of Concepts | 15% | Effectively applies relevant concepts to analyze and resolve errors | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply concepts |
| Clarity & Presentation | 10% | Presents analysis clearly, logically, and systematically | Generally clear with minor issues | Lacks clarity or proper structure | Poor presentation and difficult to understand |

# Answers

## Question 1: ECP and BVA for an Online Shopping System

**1. Error identification.** (i) Class C1 = {0 … 60} mixes valid values (1–50) with invalid ones (0, 51–60), which breaks the rule that all members of a class behave alike. (ii) The only invalid class is "negative"; values above 50, non-numeric, decimal and blank inputs have no class. (iii) No boundary values (0, 1, 50, 51) are selected, so limit faults such as < coded instead of <= stay hidden.

**2. Root cause analysis.** The tester partitioned by the sign of the number instead of by the accept/reject rule in the specification. The valid and invalid domains were never derived, and the classes were not reviewed for overlap or completeness. ECP was also used alone, although defects cluster at edges where off-by-one mistakes are made.




**3. Corrected test cases.**

| ID | Input | Class / technique | Expected result |
|---|---|---|---|
| TC1 | 25 | V1 valid (1–50) | Accepted |
| TC2, TC3 | -5 ; 75 | I1 below range ; I2 above range | Rejected with message |
| TC4–TC6 | abc ; (blank) ; 2.5 | I3 non-numeric, I4 empty, I5 decimal | Rejected with message |
| TC7–TC12 | 0, 1, 2, 49, 50, 51 | BVA around both limits | Reject, accept ×4, reject |

**4. Justification.** Every class now has one behaviour, so one value represents it, and all six classes together cover the whole input domain. The six boundary tests expose off-by-one and wrong-operator defects, so detection improves with only 12 repeatable tests.

**5. Application.** In a live cart, a missing upper-limit check lets 51 or more items reach stock and payment logic, while a wrongly rejected 50 loses a sale. The boundary tests TC7–TC12 expose either defect in a single run.

# Question 2: Random Testing in a Banking Application

**1. Error identification.** (i) "High coverage" is claimed without any measure, so no test is linked to a requirement or to code. (ii) Random inputs rarely hit limits: 100 random amounts from Rs.0–1,00,000 have only about a 0.1% chance of including exactly Rs.50,000. (iii) No expected results were planned, so wrong outputs can pass. (iv) Combinations (limit, daily total, account status) and states (blocked account) were never designed.

**2. Root cause analysis.** The number of tests was confused with coverage. Requirements were not analysed and risk was not prioritised, even though money limits are high-risk. Nobody applied the knowledge that defects cluster at boundaries, and random testing was used as the main strategy instead of a supplement.

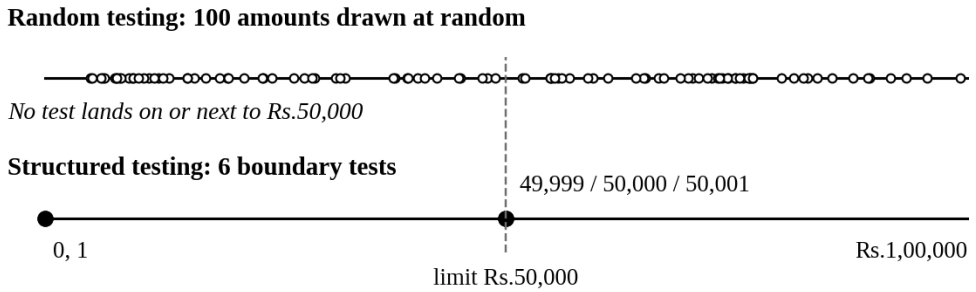



*Figure 2: Where random and structured tests fall on the amount scale.*

**3. Corrected approach.**

| Technique | Application to the bank | Defect targeted |
|---|---|---|
| ECP | < 1, 1–50,000, > 50,000, non-numeric | Missing validation |
| BVA | 0, 1, 2, 49,999, 50,000, 50,001 | Off-by-one limit errors |
| Decision table | Within limit × daily balance × account active | Wrong approve / reject |
| State transition | Active → Blocked → Active | Payments on blocked account |

**4. Justification.** Each technique gives a countable target (every class, boundary and rule). Coverage becomes measurable, tests are repeatable for regression, and limit defects are found deterministically rather than by luck. Random testing stays only as a final robustness check.

**5. Application.** If the limit is coded as < instead of <=, a legitimate transfer of Rs.50,000 is refused. 100 random amounts almost never expose this, but the boundary test at 50,000 finds it immediately.

# Question 3: Control Flow Graph and Path Testing

**1. Error identification.** The graph is drawn as straight-line code. (i) The loop-back edge 6→2 is missing, so repetition is not modelled. (ii) Decision node 2 has no False exit to node 7. (iii) The else node 5 is missing, so only one outcome of each decision exists. V(G) falls to 1, meaning one path of three is tested (about 33% path coverage).

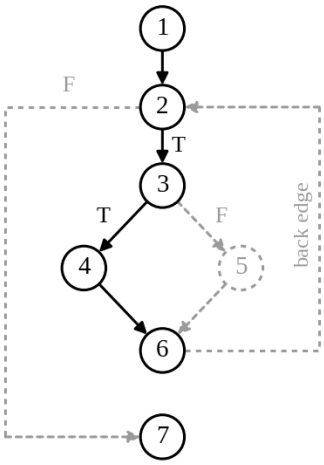
**2. Root cause analysis.** Nodes were drawn from the "happy path" reading of the code. The predicates `while` and `if` were not recognised as nodes with two outgoing edges, source lines were not numbered, so `else` was overlooked, and V(G) was never computed as a cross-check or peer-reviewed.

**Code under test**

```
1  read(n); sum=0; cnt=0; i=1;
2  while (i <= n) {
3      if (x[i] > 0)
4          sum = sum + x[i];
5      else cnt = cnt + 1;
6      i = i + 1; }
7  print(sum, cnt);
```

*Nodes 1 to 7 = numbered statements / predicates*

**Incomplete CFG (as drawn)**




**Corrected CFG**

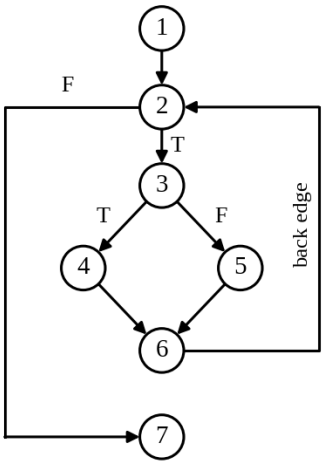



*Figure 3: Code, incomplete CFG and corrected CFG (dotted grey = missing).*

**3. Correction.** N = 7, E = 8, so V(G) = E − N + 2 = 3, which equals predicates + 1 (2 + 1).

| Path | Node sequence | Test case |
|---|---|---|
| P1 | 1-2-7 (loop skipped) | n = 0 |
| P2 | 1-2-3-4-6-2-7 | n = 1, x[1] = 5 (positive) |
| P3 | 1-2-3-5-6-2-7 | n = 1, x[1] = -2 (non-positive) |

**4. Justification.** Executing all independent paths covers every edge and statement, giving 100% branch coverage with only V(G) tests. It reveals loop-skip faults (n = 0), wrong-branch faults and an uninitialised `cnt`.

**5. Application.** The three paths form a basis set. A mutant that forgets `i = i + 1` or starts the loop at i = 0 is caught by P1–P3, whereas the one-path suite from the incomplete graph passes silently.

# Question 4: Statement Coverage versus Logical Errors

**1. Error identification.** One test (total = 2000) runs every statement, so statement coverage is 100%, yet the False outcome of the `if` never runs (branch coverage 50%). The boundary total = 1000 is mishandled (> instead of >=), and nobody checks that small orders get no discount.

**2. Root cause analysis.** Statement coverage only proves that lines executed, not that each decision outcome was taken or that results were correct. An implicit else has no statement to count. Relational-operator and boundary faults need specific input values. The team also chased a metric ("100%") instead of risk and did not verify expected results.

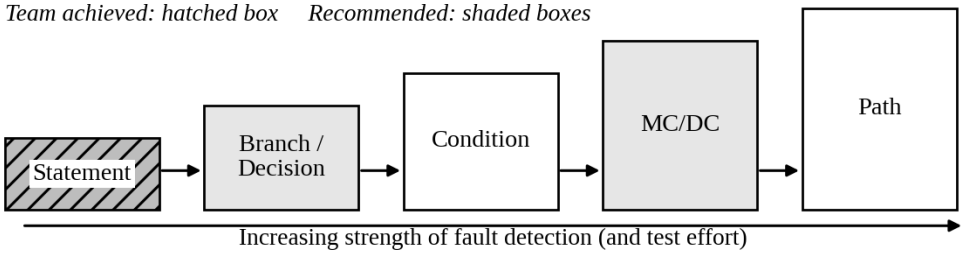



*Figure 4: Hierarchy of white-box coverage criteria.*

**3. Recommended strategy.** Use branch coverage as the minimum, MC/DC for compound conditions, boundary values for each predicate, and path coverage for critical modules.

| Test | Total | Stmt | Branch | Defect revealed |
|---|---|---|---|---|
| T1 | 2000 | 100% | 50% | None (original suite) |
| T2 | 500 | 100% | 100% | Discount wrongly applied to small orders |
| T3 | 1000 | 100% | 100% | > vs >= (expected 900, got 1000) |

**4. Justification.** Branch coverage forces both outcomes of every decision, and the boundary test targets the operator fault itself, so logical errors surface that line coverage cannot see.

**5. Application.** The same reasoning applies to tax, eligibility and discount rules: wherever a decision guards business logic, report branch and boundary coverage next to statement coverage so that a high percentage is not mistaken for correctness.

# Question 5: Equivalence Partitioning for Age Validation (0–100)

**1. Error identification.** (i) "Positive numbers" merges valid ages (1–100) with invalid ones (101, 500). (ii) Zero is neither negative nor positive, so a valid age falls outside both classes and the partition is incomplete. (iii) Invalid classes for age > 100, non-numeric, blank and decimal input are missing. (iv) No boundary values (-1, 0, 1, 99, 100, 101) are tested.

**2. Root cause analysis.** The tester partitioned by mathematical sign rather than by the specified interval [0, 100]. The valid/invalid distinction was ignored, the rules of a good partition (disjoint, complete, homogeneous) were not checked, and BVA was not used as a companion technique.

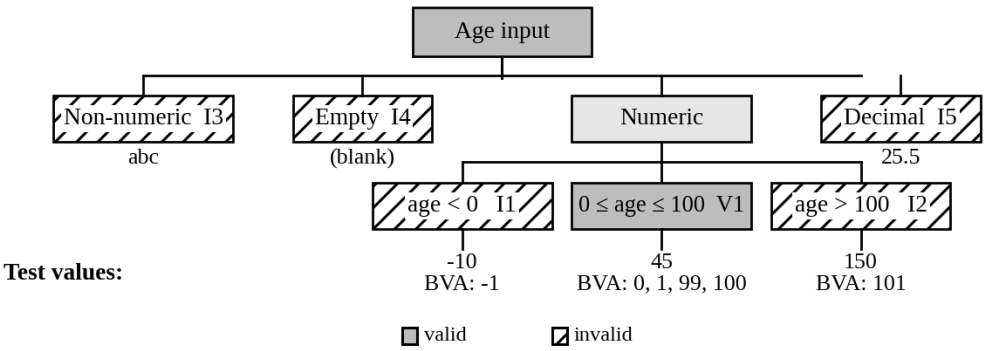



*Figure 5: Corrected input-domain partition tree with test values.*

**3. Corrected partitions.**

| Class | Range | Representative | Boundary tests |
|---|---|---|---|
| V1 | 0 ≤ age ≤ 100 (valid) | 45 | 0, 1, 99, 100 → accept |
| I1 / I2 | age < 0 / age > 100 | -10 / 150 | -1 / 101 → reject |
| I3–I5 | non-numeric, blank, decimal | abc, (blank), 25.5 | Reject with message |

**4. Justification.** The classes are disjoint, complete and homogeneous, so each is tested once. The 12 resulting tests (6 representatives, 6 boundaries) catch faults such as `age < 100` or a missing type check, keeping invalid data out of the system.

**5. Application.** In a registration form, accepting 101 or 0 as an age can break eligibility or insurance rules downstream. Testing exactly 0, 100 and 101 shows where validation switches from accept to reject.

# Question 6: Condition Coverage of a Compound Decision

**1. Error identification.** Condition coverage only needs each of A, B, C to be True and False once. Tests (T,F,T) → True and (F,T,F) → False achieve this, so the claim looks valid, but (i) the effect of each condition on the outcome is never shown, (ii) with short-circuit evaluation B is not even evaluated when A is False, and (iii) the faulty version `A || C` (B omitted) gives identical results on both tests, so that fault is missed.

**2. Root cause analysis.** Coverage of condition values was confused with coverage of their influence on the decision. Masking (C = True hides A and B) and short-circuiting were ignored, and tests were picked arbitrarily instead of from the truth table.

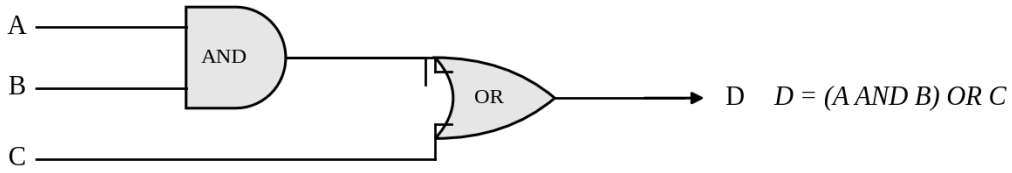



*Figure 6: Logic structure of the decision under test.*

**3. Corrected test set (MC/DC).** Each condition must change the outcome while the others stay fixed.

| Test | A | B | C | Outcome | Independence shown |
|---|---|---|---|---|---|
| T1 | T | T | F | True | A (with T2) and B (with T3) |
| T2 | F | T | F | False | A changes the outcome |
| T3 | T | F | F | False | B (with T1) and C (with T4) |
| T4 | T | F | T | True | C changes the outcome |

**4. Justification.** MC/DC proves each condition independently affects the decision. It needs only n + 1 = 4 tests instead of $2^3 = 8$, and T3 returns True under `A || C` where False is expected, exposing the fault.

**5. Application.** Authorisation checks such as `(isAdmin && isActive) || isOwner` follow the same pattern. MC/DC shows that dropping `isActive` changes behaviour, which condition coverage alone would never reveal.

# Question 7: Decision Table Testing for a Discount System

**1. Error identification.** The table held only three rules: (Premium, Coupon), (Premium, No coupon) and (Non-premium, No coupon). The combination Non-premium + Coupon is missing, so only 3 of 4 combinations are covered. "Coupon valid" was also not a condition, and no action was defined for an invalid coupon.

**2. Root cause analysis.** The table was built from examples quoted in the requirements instead of enumerating all $2^n$ combinations. Coupon validity was hidden inside "Coupon Applied", no completeness check was made (rule count after expansion must equal $2^n$), and non-members with coupons were treated as a default case. This causes errors such as coupons silently ignored or discounts stacked wrongly.

**Original table: 3 of 4 combinations**

| Premium \ Coupon applied | Yes | No |
|---|---|---|
| Yes | covered | covered |
| No | MISSING | covered |

**Revised table: 4 of 4 combinations**

| Premium \ Coupon applied | Yes | No |
|---|---|---|
| Yes | covered | covered |
| No | covered | covered |

**3. Revised exhaustive decision table** ("–" = not applicable).

| Condition / action | R1 | R2 | R3 | R4 | R5 | R6 |
|---|---|---|---|---|---|---|
| Premium Member | Y | Y | Y | N | N | N |
| Coupon Applied | Y | Y | N | Y | Y | N |
| Coupon Valid | Y | N | – | Y | N | – |
| Show error message | – | X | – | – | X | – |
| Total discount | 15% | 10% | 10% | 5% | 0% | 0% |
| Price on Rs.2,000 | 1,700 | 1,800 | 1,800 | 1,900 | 2,000 | 2,000 |

**4. Justification.** The six rules expand to $1 + 1 + 2 + 1 + 1 + 2 = 8 = 2^3$ combinations with no gaps or overlaps. Each rule gives one test with an expected price, so wrong stacking, ignored coupons and accepted invalid coupons are caught before release.

**5. Application.** For a Rs.2,000 cart, a non-member with a valid coupon should pay Rs.1,900. The old table had no such rule, so the system could charge Rs.2,000 or fail; R4 now fixes the expected result and becomes a regression test.

# Question 8: Loop Testing Strategy

**1. Error identification.** Only n = 1 and n = max were run. Missed: n = 0 (loop skipped, so initialisation and empty-input faults such as division by zero remain hidden), n = max + 1 (overflow or <= versus < mistakes), and n = 2, a typical m and max − 1 (faults caused by data carried between iterations). Nested loops were not tested systematically.

**2. Root cause analysis.** The tester assumed errors occur only at the first and last pass, ignoring loop entry and exit conditions. The loop control variable was not treated as a boundary domain, the zero, one and many-iteration paths were not reasoned about, and the cost of nested combinations ($max^2$) led to arbitrary selection.

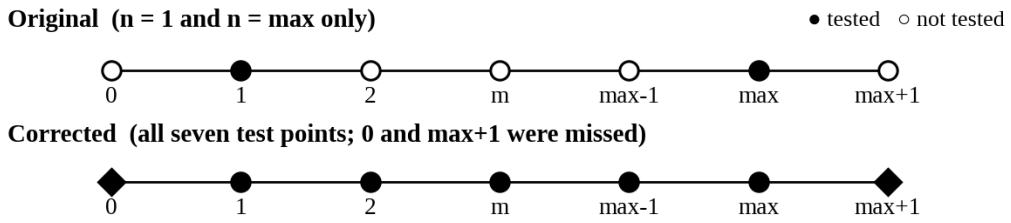



*Figure 8: Iteration counts tested originally and after correction.*

**3. Corrected approach.**

| Loop type | Test strategy | Tests |
|---|---|---|
| Simple | n = 0, 1, 2, m, max − 1, max, max + 1 | 7 |
| Nested | Innermost first with outer loops at minimum; then move outward with inner loops at typical values; finally all at typical, then all at max | ≈ 13 (not 49) |
| Concatenated | Independent: test as simple loops; dependent: treat as nested | 7 each |

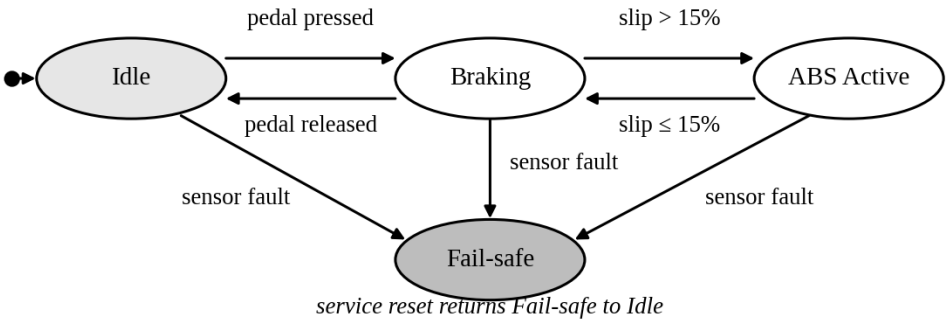
**4. Justification.** The set exercises every loop path class (skip, once, many, limit, beyond limit), so entry, back edge and exit are all proven. Nested tests grow linearly rather than exponentially, keeping path integrity affordable.

**5. Application.** In a billing loop over order items, an empty cart (n = 0) and an over-limit cart (n = max + 1) are the real-world cases that crash systems, so they are tested first rather than last.

## Question 9: Error Guessing versus Structured Braking Tests

**1. Error identification.** Relying on error guessing means (i) test selection depends on one engineer's memory, so coverage is unsystematic, (ii) tests are not traceable to safety requirements, as safety standards such as ISO 26262 expect, (iii) state-sequence faults (for example pedal release during ABS) are rarely guessed, and (iv) results are neither repeatable nor measurable.

**2. Impact on test adequacy.** Input ranges and states remain unexplored, threshold faults at the ABS slip limit can escape, and residual risk is unknown. There is no objective exit criterion, so safety cannot be argued, which is unacceptable where a missed defect means a hazard.




**3. Corrected strategy.** *BVA:* slip ratio 14%, 15%, 16%; ABS-disable speed 4, 5, 6 km/h; pedal input 0%, 1%, 99%, 100%. *State transition:* TC1 Idle→Braking, TC2 Braking→ABS, TC3 ABS→Braking, TC4 Braking→Idle, TC5 any state→Fail-safe, plus invalid events (TC6: slip above 15% while Idle) which must cause no state change. Error guessing remains only as a supplement.

**4. Justification.** BVA finds threshold faults deterministically and state testing covers every state and transition, valid and invalid. Together they are traceable and measurable, giving objective evidence and higher safety assurance.

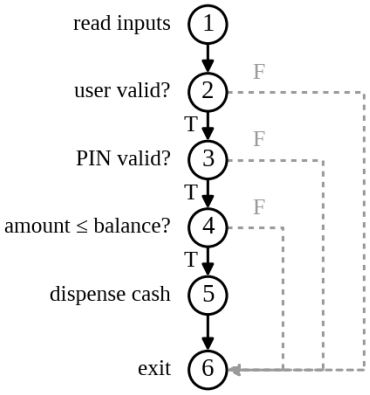
**5. Application.** If the ABS threshold is coded as `slip >= 15` instead of `> 15`, the 15% boundary test fails at once, and TC3 shows whether ABS releases correctly, a sequence that guessing seldom reaches.

# Question 10: Control Flow Graph with Nested if Statements

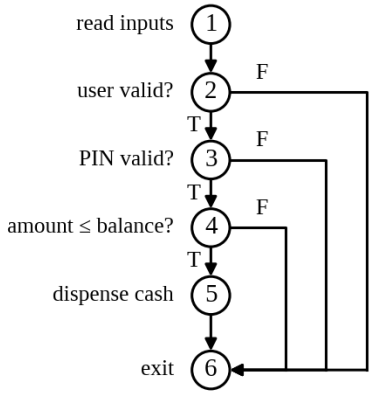
**1. Error identification.** Only True edges were drawn, so nodes 2, 3 and 4 each have a single exit. The three False edges to the exit node 6 are missing. One test (all valid) touches every node, giving 100% statement coverage but only 3 of 6 branch outcomes (50%), and 0% on the error paths. V(G) was taken as 1 instead of 4.

**2. Root cause analysis.** An implicit else has no statement to attach a node to, so the graph was built from statements instead of predicates. Error handling was treated as optional, V(G) = P + 1 was not used as a cross-check, and the coverage tool measured statements only.

**Incomplete CFG (as drawn)**




**Corrected CFG**




**3. Correction.** N = 6, E = 8, V(G) = 8 − 6 + 2 = 4 = 3 + 1. Independent paths: **P1** 1-2-3-4-5-6 (success), **P2** 1-2-6 (invalid user), **P3** 1-2-3-6 (invalid PIN), **P4** 1-2-3-4-6 (insufficient balance).

**4. Justification.** P1 alone gives 3/6 = 50% branch coverage; the four paths together execute all six decision outcomes (100%). The security-critical error paths are tested, and V(G) confirms that four tests are sufficient.

**5. Application.** Reporting branch coverage next to statement coverage in code reviews would have exposed the untested rejection paths at once.